Machine Learning A (2026)
Home Assignment 1

Casper Moes, hwd362

Contents

1 Make Your Own (10 points) **2**

2 Digits Classification with K Nearest Neighbors (40 points) **3**

3 Regression (50 points) **6**

3.1 Task 1 (5 points) 6

3.2 Task 2 (10 points) 6

3.3 Task 3 (10 points) 7

3.4 Task 4 (5 points) 7

3.5 Task 5 (10 points) 8

3.6 Task 6 (10 points) 9

1 Make Your Own (10 points)

1. Apart from the dimensions named in the assignment description the sample space $\mathcal{X}$ could also contain time spend studying during the course and number of lectures attended.
2. The label space contains the possible grades: $\mathcal{Y} \in \{-3, 00, 02, 4, 7, 10, 12\}$.
3. One possible loss function could be the squared error loss defined by $\ell(y, \hat{y}) = (y - \hat{y})^2$, which penalizes large mistakes harshly reflecting that very unfair / wrong grades should be avoided.
4. One could use the following metric where $x, x' \in \mathcal{X}$ are unlabeled samples: $d(x, x') = \|x - x'\|_2$. This fulfills $d(a, a) = 0$, $d(a, b) = d(b, a)$, $d(a, b) \geq 0$ and the triangle inequality.
5. To evaluate the performance of the hypothesis / algorithm h i would set aside a test set $T \subseteq S$ with $n < N$ elements, where $S = \{(X_1, Y_1), \ldots, (X_N, Y_N)\}$ contains all samples / data. I would then calculate 1.1, the empirical risk of the model

$$\mathcal{R}_T(h) = \frac{1}{n} \sum_{i=1}^{n} \ell\left(Y_i, h(X_i)\right), \quad (X_i, Y_i) \in T. \tag{1.1}$$

This is an unbiased estimate of the true risk $\mathcal{R}_p(h)$ when $h \perp\!\!\!\perp T$ holds (i.e. that the hypothesis is independent of the test data) and all data points in S are drawn from the same distribution p. This is quick to show:

$$\begin{aligned}
\mathbb{E}\left[\mathcal{R}_T(h)\right] &= \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^{n} \ell\left(Y_i, h(X_i)\right)\right] \\
&= \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}\left[\ell\left(Y_i, h(X_i)\right)\right] \\
&= \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}\left[\mathbb{E}\left[\ell\left(Y_i, h(X_i)\right) \mid h\right]\right] \\
&\overset{h \perp\!\!\!\perp T}{=} \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}_{(X,Y)\sim p}\left[\ell\left(Y, h(X)\right)\right] \\
&= \frac{1}{n} \sum_{i=1}^{n} \mathbb{E}_{(X,Y)\sim p}\left[\ell\left(Y, h(X)\right)\right] \\
&= \mathbb{E}_{(X,Y)\sim p}\left[\ell\left(Y, h(X)\right)\right] = \mathcal{R}_p(h).
\end{aligned}$$

The crucial point is that independence of the hypothesis and the

training data implies that the distribution of (X_i, Y_i), $i \in T$ given h is still just p.

6. The students might learn about the models inner workings and start to trick it. For example by studying for long hours, but with very low intensity. Goodhart's law sums it up: *When a measure becomes a target, it ceases to be a good measure.*

2 Digits Classification with K Nearest Neighbors (40 points)

The vectorized K-NN function implemented in python is shown verbatim here (where np refers to numpy)

```
# knn function
def knn(train_points, train_labels, test_points,
    test_labels, seed=56):

    rng = np.random.default_rng(seed)

    m = len(train_labels) # number of training datapoints
    n = len(test_labels) # number of test datapoints

    # distance matrix
    train_sqn  = np.diag(train_points.T @ train_points).
    reshape(m,1)
    test_sqn   = np.diag(test_points.T @ test_points).
    reshape(n,1)
    trainTtest = training_points.T @ test_points
    ones = lambda q: np.ones(q).reshape(q,1)

    D = (
        train_sqn @ ones(n).T
        - 2*trainTtest
        + ones(m) @ test_sqn.T
    )

    # distance sorting
    D_sort_index  = np.argsort(D,axis=0)
    sorted_labels = training_labels[D_sort_index]

    # predictions
    sorted_labels_cumsum= sorted_labels.cumsum(axis=0)
    pred = np.sign(sorted_labels_cumsum)
```

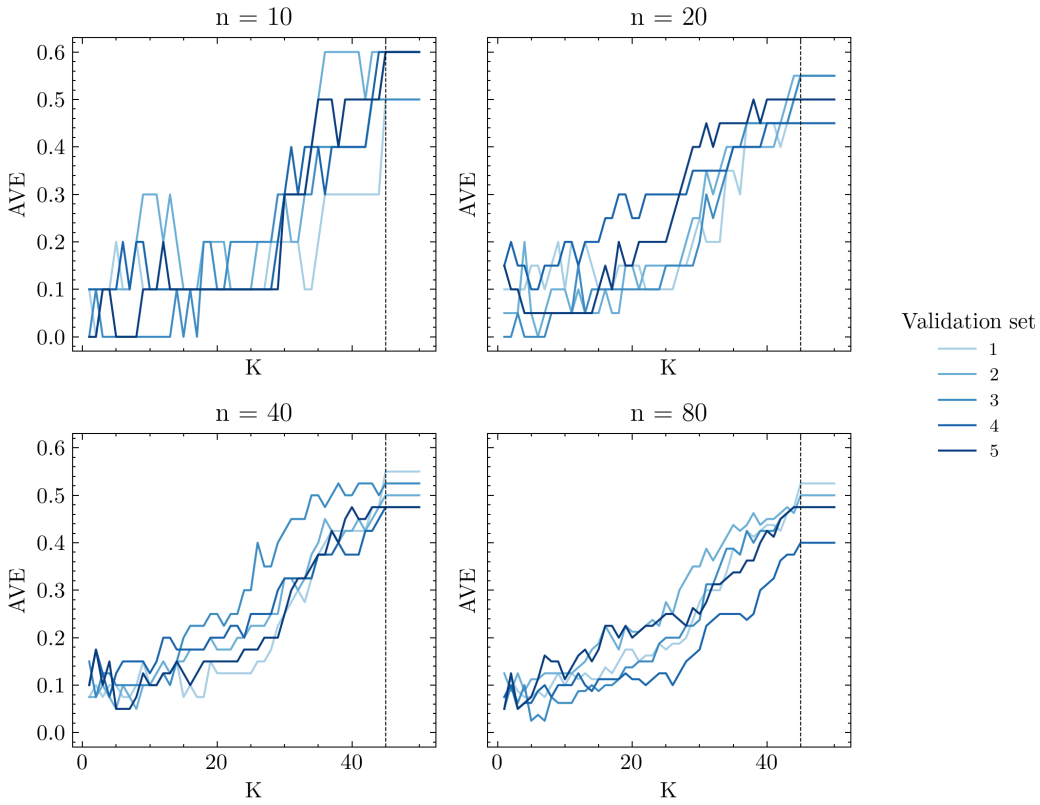


Figure 1: Average validation error for different sizes of validation sets

Note: The lines display the average validation error as a function of K. They are colored based on the validation set which are defined as in the description of the assignment. n defines the size of each of the corresponding plots validation sets.

```
    # randomly assign -1 or 1 wherever there is a tie
    ties = pred == 0
    pred[ties] = rng.choice([-1, 1], size=ties.sum())

    # predictions errors and average errors
    error     = pred != test_labels
    avg_error = np.mean(error,axis=1)

    return avg_error
```

The resulting average validation error (AVE) for the different models and validation sets are shown in figure (1).

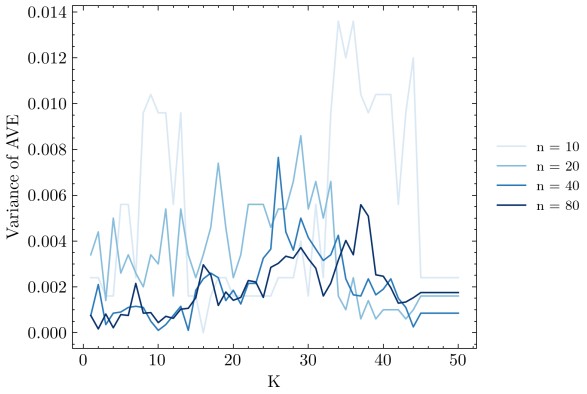


Figure 2: Variance of validation error

Note: The lines plot the empirical variance of the AVE over the 5 validation sets for different K.

From this figure we see that larger K are associated with larger AVE and that larger validation sets (i.e. larger m) results in more granular AVE-curves. In addition to this figure (2) shows how fluctuations of the AVE tends to decrease as a function of n.

Note that the prediction of K-NN trained on these data (i.e. the first $m = 50$ digits of the provided data) will always predict $h_K(\boldsymbol{x}_i) = 1$ (i.e. the digit 6) for $K \geq 45$. This is simply because 22 out of the 50 digits have 5 as their label so that

$$\sum_{k=1}^{K} S_k^* > 0, \quad K \geq 45,$$

where S^*contains the K nearest neighbors to the input $\boldsymbol{x}_i$. And therefore

$$h_K(\mathbf{x}_i) = \begin{cases} -1, & \text{if } \sum_{k=1}^{K} S_k^* < 0 \\ 1, & \text{if } \sum_{k=1}^{K} S_k^* > 0 \\ \text{chosen uniformly at random from } \{-1, 1\}, & \text{if } \sum_{k=1}^{K} S_k^* = 0. \end{cases}$$

$= 1,$ for $K \geq 45$.

3 Regression (50 points)

3.1 Task 1 (5 points)

```
def reg(X,y):
    # assuming X is N x (k+1)

    # estimating the parameters
    b = (
        np.linalg.inv(X.T @ X) @
        X.T @ y
    )

    # calculating predicted PCB values
    y_pred = np.exp(X @ b)
    y_raw  = np.exp(y)

    # calculating MSE and R^2
    MSE = np.mean((y_raw-y_pred)**2)
    R2 = 1 - MSE / np.sum((y_raw-np.mean(y_raw))**2)

    return b, MSE, R2
```

3.2 Task 2 (10 points)

I estimate model (3.1) using OLS

$$\ln(y_i) = b + ax_i + u_i, \quad i \in \{1, 2, \ldots, n\}, \tag{3.1}$$

where $u_i \equiv \ln(y_i) - (b + ax_i)$ defines the error term for observation i in the dataset $S = \{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$. The results are summarized in table 1.

	$\hat{a}$	$\hat{b}$	MSE
Value	0.031	0.259	34.84

Table 1: OLS estimatation results for model (3.1)

3.3 Task 3 (10 points)

Let the two point be $\{(x_1, y_1), (x_2, y_2)\} = \{(0, 2), (0, 1)\}$ then the first problem is

$$\begin{aligned} b^* &= \arg\min_{a,b} \sum_{i=1}^{2} (y_i - h(x_i))^2 \\ &= \arg\min_{b} \left\{ (2 - \exp(b))^2 + (1 - \exp(b))^2 \right\}, \end{aligned}$$

with the first order condition (FOC) yielding b^*

$$\begin{aligned} -2\left(2 - \exp(b^*)\right)\exp(b^*) - 2\left(1 - \exp(b^*)\right)\exp(b^*) \\ = -2\exp(b^*)\left(3 - 2\exp(b^*)\right) = 0 \iff \\ b^* = \ln\frac{3}{2}. \end{aligned}$$

The second problem is

$$\begin{aligned} \beta^* &= \arg\min_{\alpha,\beta} \sum_{i=1}^{2} (\ln y_i - (\alpha x_i + \beta))^2 \\ &= \arg\min_{\beta} \left\{ (\ln 2 - \beta)^2 + (\ln 1 - \beta)^2 \right\}, \end{aligned}$$

with the FOC yielding β^*

$$\begin{aligned} -2\left(\ln 2 - \beta^*\right) - 2\left(\ln 1 - \beta^*\right) = -2\left(\ln 2 - 2\beta^*\right) = 0 \iff \\ \beta^* = \frac{\ln 2}{2} = \ln\sqrt{2}. \end{aligned}$$

Since $\sqrt{2}$ is an irrational number, and thus not equal to $3/2$, and $\ln(\cdot)$ is a strictly increasing function, and thus one-to-one, we can safely state that $b^* \neq \beta^*$.

3.4 Task 4 (5 points)

The resulting plot is shown in figure 3

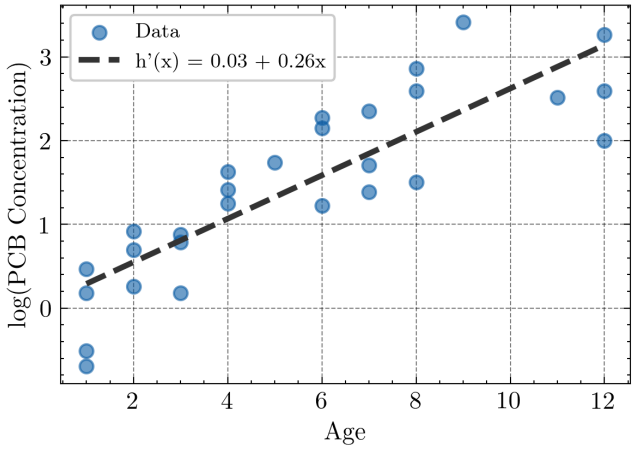


Figure 3: Visualizing the fitted model (3.1)

Note: The figure plots the transformed PCB datapoints along with the model predictions estimated using OLS.

3.5 Task 5 (10 points)

The quantity R^2 measure how well the estimated hypothesis h predicts the labels y_i. For the estimated model h we have $R^2 = 0.977$. We see that for larger R^2 the loss $\ell\left(y_i, h(x_i)\right) = \left(y_i - h(x_i)\right)^2$ is generally smaller. In other words the empirical risk is smaller. For $R^2 = 1$ in particular:

$$R^2 = 1 - \frac{\sum_{i=1}^{N}\left(y_i - h(x_i)\right)^2}{\sum_{i=1}^{N}\left(y_i - \bar{y}\right)^2} = 1 \iff 0 = \sum_{i=1}^{N}\left(y_i - h(x_i)\right)^2$$
$$\iff h(x_i) = y_i,\ (x_i, y_i) \in S.$$

That is, the hypothesis perfectly predicts the label value for all observations in S, which for example could be the training data. Reversely lower values of R^2 indicates larger empirical risk with

$$R^2 = 1 - \frac{\sum_{i=1}^{N}\left(y_i - h(x_i)\right)^2}{\sum_{i=1}^{N}\left(y_i - \bar{y}\right)^2} = 0 \iff \sum_{i=1}^{N}(y_i - \bar{y})^2 = \sum_{i=1}^{N}\left(y_i - h(x_i)\right)^2,$$

where h has the same empirical risk as the hypothetical model $h^2(x_i) = \bar{y}$ that always predicts the average output value of the data.

Note that R^2 can be negative, which would indicate a hypothesis / model that performs worse than a simple average under the chosen squared difference loss function.

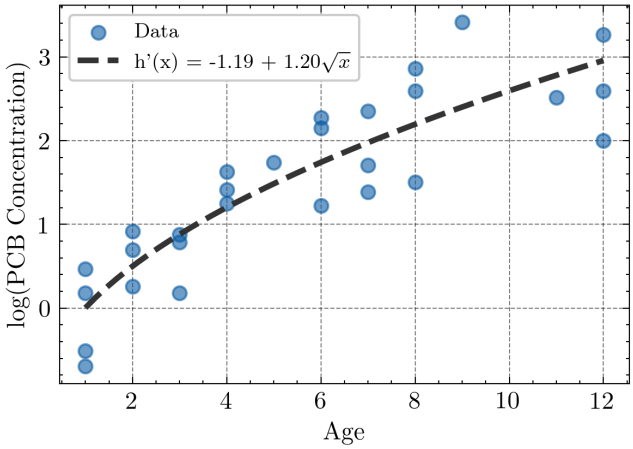


Figure 4: Visualizing the fitted model (3.2)

Note: The figure plots the transformed data along with the model predictions estimated using OLS.

3.6 Task 6 (10 points)

Now we turn to model 3.2

$$\ln(y_i) = b + a\sqrt{x_i} + u_i, \quad i \in \{1, 2, \ldots, n\}. \tag{3.2}$$

The estimation results are shown in tabel 2 and figure 4.

	$\hat{a}$	$\hat{b}$	MSE	R^2
Value	-1.195	1.200	28.084	0.982

Table 2: OLS estimatation results for model (3.2)